

CS 598 Homework 1

Ilia Kheirkhah
Parallel Programming

September 9, 2026

Problem 1 - Matrix Multiplication Optimization

In this problem, we will optimize a matrix multiplication algorithm. The results of iterations are shown below. Before optimizing the algorithm, I implemented a driver that will run the method and create an output file for analysis. This driver is included here:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  /* Matrix multiplication function from mxm.c */
6  void dgemm_kernel(
7      int m,
8      int n,
9      int k,
10     const double *A,
11     const double *B,
12     double *C
13 );
14
15 int main(void)
16 {
17     /* Open output file */
18     FILE *log = fopen("benchmark.log", "w");
19
20     if (log == NULL) {
21         return 1;
22     }
23     printf("%10s %15s %15s\n", "N", "Time", "Gflops");
24     fflush(log);
25     for (int N = 8; N <= 2000; N += 32) {
26
27
28         /* Allocate matrices */
29         double *A = malloc(N * N * sizeof(double));
30         double *B = malloc(N * N * sizeof(double));
31         double *C = malloc(N * N * sizeof(double));
32
33         /* Fill A and B with random numbers */
34         for (int i = 0; i < N * N; ++i) {
35             A[i] = (double)rand() / RAND_MAX;
36             B[i] = (double)rand() / RAND_MAX;
37         }
38
39         /* Start timer */
40         struct timespec start, end;
41
42
```

```

43     clock_gettime(CLOCK_MONOTONIC, &start);
44
45
46     /* C = A * B 10 times */
47     for (int i = 0; i < 10; ++i) {
48         dgemm_kernel(N, N, N, A, B, C);
49     }
50
51     /* Stop timer */
52     clock_gettime(CLOCK_MONOTONIC, &end);
53
54     double time =
55         (end.tv_sec - start.tv_sec) +
56         (end.tv_nsec - start.tv_nsec) * 1e-9;
57
58     time /= 10.0;
59     /* COMPUTE GLOPS */
60     int N3 = N * N * N;
61     double gflops = (2.0 * N3 / time) / 1e9;
62     /* Write to log file */
63     fprintf(log, "%d %f %f\n", N, time, gflops);
64     printf("%d %f %f\n", N, time, gflops);
65     fflush(log);
66     /* Free memory */
67     free(A);
68     free(B);
69     free(C);
70 }
71 fclose(log);
72
73 return 0;
74 }

```

In addition, we benchmarked my Macbook by using the provided matlab file to get the graph we want and measure speeds of an established code. The results are included here.

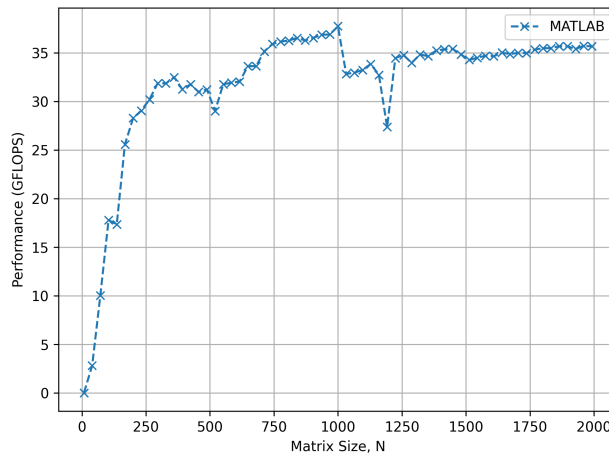


Figure 1: MATLAB Benchmark

The peak here is around 38 GLOPS. We want to achieve a speed at least $\frac{2}{3}$ of this, which is about 25.3 GFLOPS. This will be achieved over a series of iterations, detailed below.

Baseline

To begin, we wrote a simple and naive implementation of matrix multiplication. This looks like follows:

```
1 void dgemm_kernel(  
2     int m,  
3     int n,  
4     int k,  
5     const double *A,  
6     const double *B,  
7     double *C )  
8 {  
9     int i,j,p;  
10    for(j=0;j<n;++j) {  
11        for (i=0;i<m;++i) {  
12            double cij = 0.0;  
13            for (p=0;p<k;++p) {  
14                cij += A[i + p*m] * B[p + j*k];  
15            }  
16            C[i + j*m] = cij;  
17        }  
18    }  
19 }
```

We then compiled this with a make file that looks like:

```
1 CC = gcc  
2 CFLAGS = -O3 -march=native -std=c99 -Wall -Wextra  
3 OBJS = bench.o mxm.o  
4 bench: $(OBJS)  
5         $(CC) $(CFLAGS) -o bench $(OBJS)  
6 clean:  
7         rm -f *.o bench
```

The resulting executable can be run and outputs a datafile we can analyze. Plotting the resulting speeds are shown below

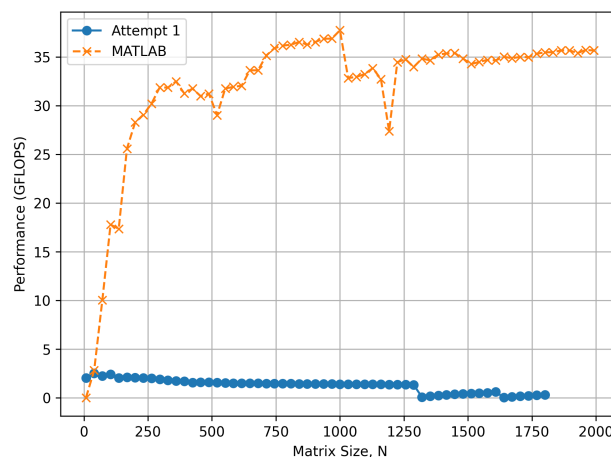


Figure 2: Baseline

The peak GFLOPS achieved in this test was 2.5 GFLOPS at $N = 40$. This is reasonable, as there are a noable amount of issues that we will resolve as we go.

Attempt 2 - Strides

The first change I made was change the loop indexing to go in stride with the array indexing used. Before, we would be jumping around in the memory. By changing the ordering of the loops, we can access adjacent locations in memory often, soemtimes taking advantage of cache storage. Below is the implementation. The make file has not changed.

```

1  /*
2  From previous attempt:
3  Change Loop order to j, p, i to improve cache performance.
4  Striding is better.
5  */
6  void dgemm_kernel(
7      int m,
8      int n,
9      int k,
10     const double *A,
11     const double *B,
12     double *C )
13 {
14     int i,j,p;
15     /* Zero C */
16     for (j = 0; j < n; ++j) {
17         for (i = 0; i < m; ++i) {
18             C[i + j*m] = 0.0;
19         }
20     }
21
22     /* Compute C = A * B */
23     for (j = 0; j < n; ++j) {
24         for (p = 0; p < k; ++p) {
25
26             double Bpj = B[p + j*k];
27
28             for (i = 0; i < m; ++i) {
29                 C[i + j*m] += A[i + p*m] * Bpj;
30             }
31         }
32     }
33 }

```

This had a substantial improvement in performance, as seen below. The new peak speed was about 9.2 GFLOPS at $N = 968$. Note that after this we added restrict keywords to the matrices, but did not see substantial improvement.

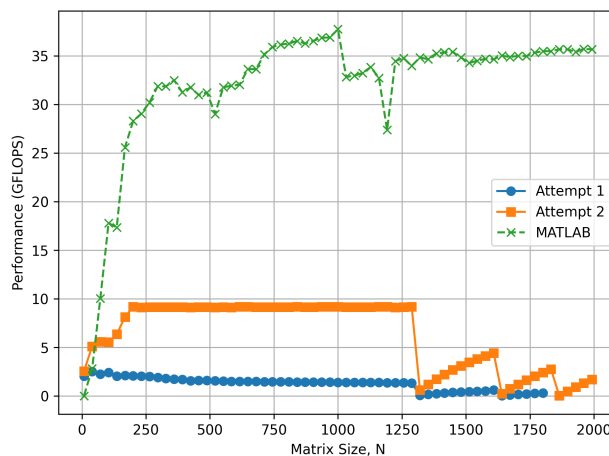


Figure 3: Proper Striding

Attempt 4 - Microkerneling

After one failed attempt of the restrict keyword, we attempted microkerneling. In this, we solve for a 4x4 block of C at a time. This allows us to pull in 4 values from both A and B which allows us to make good use of the registers we have. Doing this had a substantial improvement in the performance. Below is the code.

```
1  /*
2  From Previous Version:
3  Microkernel optimization to do 4x4 block multiplication at a time.
4
5  */
6  void dgemm_kernel(
7      int m,
8      int n,
9      int k,
10     const double * restrict A,
11     const double * restrict B,
12     double * restrict C)
13 {
14     int i, j, p;
15
16     for (j = 0; j <= n - 4; j += 4) {
17         for (i = 0; i <= m - 4; i += 4) {
18
19             double c00 = 0.0, c10 = 0.0, c20 = 0.0, c30 = 0.0;
20             double c01 = 0.0, c11 = 0.0, c21 = 0.0, c31 = 0.0;
21             double c02 = 0.0, c12 = 0.0, c22 = 0.0, c32 = 0.0;
22             double c03 = 0.0, c13 = 0.0, c23 = 0.0, c33 = 0.0;
23
24             for (p = 0; p < k; ++p) {
25
26                 double a0 = A[i + 0 + p*m];
27                 double a1 = A[i + 1 + p*m];
28                 double a2 = A[i + 2 + p*m];
29                 double a3 = A[i + 3 + p*m];
30
31                 double b0 = B[p + (j + 0)*k];
32                 double b1 = B[p + (j + 1)*k];
33                 double b2 = B[p + (j + 2)*k];
34                 double b3 = B[p + (j + 3)*k];
35
36                 c00 += a0 * b0;
37                 c10 += a1 * b0;
38                 c20 += a2 * b0;
39                 c30 += a3 * b0;
40
41                 c01 += a0 * b1;
42                 c11 += a1 * b1;
43                 c21 += a2 * b1;
44                 c31 += a3 * b1;
45
46                 c02 += a0 * b2;
47                 c12 += a1 * b2;
48                 c22 += a2 * b2;
49                 c32 += a3 * b2;
50
51                 c03 += a0 * b3;
52                 c13 += a1 * b3;
53                 c23 += a2 * b3;
54                 c33 += a3 * b3;
55             }
56
57             C[i + 0 + (j + 0)*m] = c00;
58             C[i + 1 + (j + 0)*m] = c10;
59             C[i + 2 + (j + 0)*m] = c20;
60             C[i + 3 + (j + 0)*m] = c30;
61         }

```

```

62         C[i      + (j + 1)*m] = c01;
63         C[i + 1 + (j + 1)*m] = c11;
64         C[i + 2 + (j + 1)*m] = c21;
65         C[i + 3 + (j + 1)*m] = c31;
66
67         C[i      + (j + 2)*m] = c02;
68         C[i + 1 + (j + 2)*m] = c12;
69         C[i + 2 + (j + 2)*m] = c22;
70         C[i + 3 + (j + 2)*m] = c32;
71
72         C[i      + (j + 3)*m] = c03;
73         C[i + 1 + (j + 3)*m] = c13;
74         C[i + 2 + (j + 3)*m] = c23;
75         C[i + 3 + (j + 3)*m] = c33;
76     }
77 }
78 }

```

Once again, we did not change the make file here. Running this and getting the results, we get a peak speed of about 18.4 GFLOPS at N=264. This can be seen in the graph below.

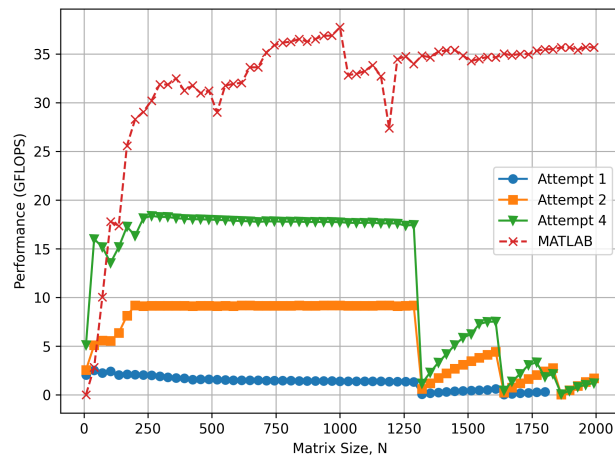


Figure 4: Proper Striding

Attempt 5 - Cache Loading

The final attempt we made was done using cache loading. The idea here is to take a block of the matrices and load it into the cache. We then do work on this section of the matrix until we have done as much useful work as we can before letting it go back to slower caches. The code is shown below.

```
1  /*
2  From Previous Version:
3  Cache blocking to maintain A,B hot while C is worked on.
4  Various BM,BN,BK Values were tried:
5  BM=64, BN=64, BK=128 = 20.078523 GFLOPS
6  BM=64, BN=64, BK=64 = 23.140657 GFLOPS
7  BM=128, BN=64, BK=64 = 23.204842 GFLOPS
8  BM=64, BN=128, BK=64 = 23.172259 GFLOPS
9  BM=32, BN=64, BK=64 = 23.113049GFLOPS
10 BM=128, BN=128, BK=64 = 23.143259 GFLOPS
11 BM=32, BN=32, BK=32 = 24.774495 GFLOPS
12 BM=16, BN=16, BK=16 = 28.939642 GFLOPS
13 BM=8, BN=8, BK=8 = 27.636178 GFLOPS
14 */
15 #define BM 16
16 #define BN 16
17 #define BK 16
18
19 void dgemm_kernel(
20     int m,
21     int n,
22     int k,
23     const double * restrict A,
24     const double * restrict B,
25     double * restrict C)
26 {
27     int i, j, p;
28     int ii, jj, kk;
29
30     /*
31      * Initialize C = 0
32      */
33     for (j = 0; j < n; ++j) {
34         for (i = 0; i < m; ++i) {
35             C[i + j*m] = 0.0;
36         }
37     }
38
39     /*
40      * Cache blocking
41      */
42     for (jj = 0; jj < n; jj += BN) {
43
44         int jend = jj + BN;
45         if (jend > n)
46             jend = n;
47
48         for (kk = 0; kk < k; kk += BK) {
49
50             int kend = kk + BK;
51             if (kend > k)
52                 kend = k;
53
54             for (ii = 0; ii < m; ii += BM) {
55
56                 int iend = ii + BM;
57                 if (iend > m)
58                     iend = m;
59
60                 /*
61                  * 4x4 microkernel
```

```

62      */
63      for (j = jj; j <= jend - 4; j += 4) {
64
65          for (i = ii; i <= iend - 4; i += 4) {
66
67              /*
68               * Load current partial C block.
69               *
70               * This matters because each kk block only
71               * computes part of the complete dot product.
72               */
73              double c00 = C[i + (j) * m];
74              double c10 = C[i + 1 + (j) * m];
75              double c20 = C[i + 2 + (j) * m];
76              double c30 = C[i + 3 + (j) * m];
77
78              double c01 = C[i + (j + 1) * m];
79              double c11 = C[i + 1 + (j + 1) * m];
80              double c21 = C[i + 2 + (j + 1) * m];
81              double c31 = C[i + 3 + (j + 1) * m];
82
83              double c02 = C[i + (j + 2) * m];
84              double c12 = C[i + 1 + (j + 2) * m];
85              double c22 = C[i + 2 + (j + 2) * m];
86              double c32 = C[i + 3 + (j + 2) * m];
87
88              double c03 = C[i + (j + 3) * m];
89              double c13 = C[i + 1 + (j + 3) * m];
90              double c23 = C[i + 2 + (j + 3) * m];
91              double c33 = C[i + 3 + (j + 3) * m];
92
93              /*
94               * Only work over the current k block.
95               */
96              for (p = kk; p < kend; ++p) {
97
98                  double a0 = A[i + p * m];
99                  double a1 = A[i + 1 + p * m];
100                 double a2 = A[i + 2 + p * m];
101                 double a3 = A[i + 3 + p * m];
102
103                 double b0 = B[p + (j) * k];
104                 double b1 = B[p + (j + 1) * k];
105                 double b2 = B[p + (j + 2) * k];
106                 double b3 = B[p + (j + 3) * k];
107
108                 c00 += a0 * b0;
109                 c10 += a1 * b0;
110                 c20 += a2 * b0;
111                 c30 += a3 * b0;
112
113                 c01 += a0 * b1;
114                 c11 += a1 * b1;
115                 c21 += a2 * b1;
116                 c31 += a3 * b1;
117
118                 c02 += a0 * b2;
119                 c12 += a1 * b2;
120                 c22 += a2 * b2;
121                 c32 += a3 * b2;
122
123                 c03 += a0 * b3;
124                 c13 += a1 * b3;
125                 c23 += a2 * b3;
126                 c33 += a3 * b3;
127             }
128
129             /*
130              * Store partial C block.
131              */

```

```

132         C[i + (j - 1) * m] = c00;
133         C[i + 1 + (j - 1) * m] = c10;
134         C[i + 2 + (j - 1) * m] = c20;
135         C[i + 3 + (j - 1) * m] = c30;
136
137         C[i + (j + 1) * m] = c01;
138         C[i + 1 + (j + 1) * m] = c11;
139         C[i + 2 + (j + 1) * m] = c21;
140         C[i + 3 + (j + 1) * m] = c31;
141
142         C[i + (j + 2) * m] = c02;
143         C[i + 1 + (j + 2) * m] = c12;
144         C[i + 2 + (j + 2) * m] = c22;
145         C[i + 3 + (j + 2) * m] = c32;
146
147         C[i + (j + 3) * m] = c03;
148         C[i + 1 + (j + 3) * m] = c13;
149         C[i + 2 + (j + 3) * m] = c23;
150         C[i + 3 + (j + 3) * m] = c33;
151     }
152 }
153 }
154 }
155 }
156 }

```

This used the same make file as before, recapped here for completion.

```

1 CC = gcc
2 CFLAGS = -O3 -march=native -std=c99 -Wall -Wextra
3 OBJS = bench.o mxm.o
4 bench: $(OBJS)
5         $(CC) $(CFLAGS) -o bench $(OBJS)
6 clean:
7         rm -f *.o bench

```

There was experimentation done to determine exactly what block sizes are best to use. This can be seen at the comments at the top of the code. After experimentation, it was found that a 16x16 sized block for all 3 matrices resulted in a substantially higher performance, so it was used for the testing. Repeatability was tested and verified. This lead to a peak speed of 28.9 GFLOPS at $N = 488$. Below the results are shown.

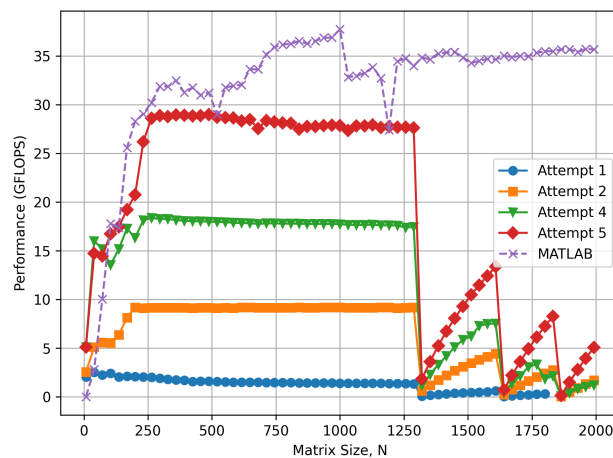


Figure 5: Proper Striding

There are a few things to note here. First, this final iteration has a speed above our target speed through changes to the C code alone. Next, there is a substantial dropoff at around $N = 1300$. Some attempts were made at changing the make file to see if that influences this, but no substantial changes to the peak or cliff were found.

Discussion

Below is the graph with our best attempt compared to MATLAB.

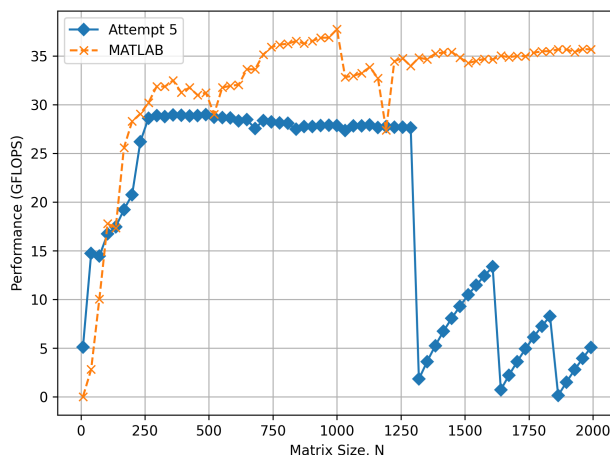


Figure 6: Best Results

For the best case, we have a $N_{1/2}$ of about 50. Compared to the first iteration, the results are about 12 times faster. The oddest part is the cliff experienced at $N = 1300$, but attempts to solve this issue did not lead to anything substantial. I am still unsure why this is happening. The interesting thing was that several components of the implementation helped substantially.

First, proper striding gave a suprising strong improvement of nearly 4x from the baseline. This shows how simple changes can lead to fairly substantial performance improvements. Next, the Microkerneling gave us another factor of 2, which although is less than I expected is still impactful for the relative ease that is associated with it. Similar things can be said for cahce loading. Being aware of when things are loaded and in memory and trying to make the most use out of what is readily available will be something that seems to be important.

Problem 2 - Core Communication

In this problem, we will implement a ping-pong test and see how well our systems perform as the message size changes. In doing this, I first created a driver that would call our ping pong method and also output a useful output file with the timing. This driver is shown below.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "msg.h"
4
5  /* Function implemented in ping_pong.c */
6  double ping_pong(void *buf, int nbytes, int p, int nloop);
7
8  int main(int argc, char **argv)
9  {
10     int rank;
11     int p;
12     int M;
13
14     int j;
15     int nwds;
16     int byte;
17     int nloop;
18
19     double msg_vol = 1000000.0;
20     double time;
21
22     FILE *fp = NULL;
23
24     msg_init(&argc, &argv);
25
26     rank = msg_rank();
27
28     /*
29      * num_ranks() returns the maximum valid rank.
30      */
31     M = num_ranks();
32
33     /*
34      * Only rank 0 opens and writes the output file.
35      */
36     if (rank == 0) {
37         fp = fopen("timing.log", "w");
38
39         if (fp == NULL) {
40             fprintf(stderr, "Failed to open timing.log\n");
41             msg_finalize();
42             return 1;
43         }
44
45         fprintf(fp, "%8s %12s %12s %12s %15s\n",
46                 "p", "nwds", "bytes", "nloop", "RTT (s)");
47     }
48
49     /*
50      * Allocate a buffer large enough for all message sizes.
51      */
52     int max_nwds = 200000;
53     double *buf = malloc(max_nwds * sizeof(double));
54
55     if (buf == NULL) {
56         fprintf(stderr, "Rank %d: Failed to allocate buffer\n", rank);
57
58         if (rank == 0)
59             fclose(fp);
60     }
```

```

61     msg_finalize();
62     return 1;
63 }
64
65 /*
66  * Test rank 0 against every available rank.
67  */
68 for (p = 0; p <= M; ++p) {
69     nwds = 0;
70
71     /*
72      * Test 500 different message sizes.
73      */
74     for (j = 0; j < 500; ++j) {
75         nwds = (int)((nwds + 1) * 1.016);
76
77         byte = 8 * nwds;
78
79         nloop = (int)(msg_vol / ((double)nwds + 2.0));
80
81         if (nloop > 1000)
82             nloop = 1000;
83
84         if (nloop < 20)
85             nloop = 20;
86
87         /*
88          * Ping-pong between rank 0 and rank p.
89          */
90         time = ping_pong(buf, byte, p, nloop);
91
92         /*
93          * Only rank 0 records the timing.
94          */
95         if (rank == 0) {
96             fprintf(fp, "%8d %12d %12d %12d %15.6e\n",
97                 p, nwds, byte, nloop, time);
98         }
99     }
100 }
101 }
102
103 if (rank == 0)
104     fclose(fp);
105
106 free(buf);
107
108 msg_finalize();
109
110 return 0;
111 }
112

```

This was amde with the makefile

```

1 CC = mpicc
2 CFLAGS = -O2 -Wall
3
4 MSG_DIR = ../../Dependencies/msg_passing_files
5
6 TARGET = ping_pong
7 SRC = driver.c ping_pong.c $(MSG_DIR)/msg_mpi.c
8
9 all: $(TARGET)
10
11 $(TARGET):
12     $(CC) $(CFLAGS) -I$(MSG_DIR) $(SRC) -o $(TARGET)
13
14 clean:
15     rm -f $(TARGET)

```

. The pingpong test is as follows:

```
1  #include "msg.h"
2
3  /*
4   * Perform nloop ping-pong exchanges between rank 0 and rank p.
5   *
6   * buf      : message buffer
7   * nbytes   : number of bytes in the message
8   * p        : partner rank
9   * nloop    : number of ping-pong repetitions
10  *
11  * Returns:
12  *   Average round-trip time per ping-pong exchange.
13  */
14  double ping_pong(void *buf, int nbytes, int p, int nloop)
15  {
16      int rank = msg_rank();
17      double t0 = 0.0;
18      double t1 = 0.0;
19      int i;
20
21      int nwarmup = 10;
22
23      /*
24       * Synchronize before warm-up.
25       */
26      msg_barrier();
27
28      /*
29       * Warm-up ping-pongs
30       */
31      for (i = 0; i < nwarmup; ++i) {
32
33          if (p == 0) { /* If (0,0) then it's a self-ping-pong */
34              if (rank == 0) {
35                  irectv(0, buf, nbytes, 100);
36                  isend(0, buf, nbytes, 100);
37                  msgwait();
38              }
39              } else { /* If (0,p) then it's a normal ping-pong */
40
41                  if (rank == 0) { /* Rank 0 is the sender */
42
43                      isend(p, buf, nbytes, 100);
44                      msgwait();
45
46                      irectv(p, buf, nbytes, 101);
47                      msgwait();
48
49                  } else if (rank == p) { /* Rank p is the receiver */
50
51                      irectv(0, buf, nbytes, 100);
52                      msgwait();
53
54                      isend(0, buf, nbytes, 101);
55                      msgwait();
56                  }
57              }
58          }
59
60      /*
61       * Make sure warm-up is completely finished.
62       */
63      msg_barrier();
64
65      /*
66       * Only rank 0 needs to time the exchange.
67       */
68      if (rank == 0)
```

```

69         t0 = msg_wtime();
70
71     for (i = 0; i < nloop; ++i) {
72
73         if (p == 0) {
74             /*
75              * Self ping-pong.
76              *
77              * Rank 0 sends a message to itself and receives it.
78              */
79             if (rank == 0) {
80                 irectv(0, buf, nbytes, 100);
81                 isend(0, buf, nbytes, 100);
82                 msgwait();
83             }
84
85         } else {
86
87             /*
88              * Normal ping-pong between rank 0 and rank p.
89              */
90             if (rank == 0) {
91
92                 /* Ping: rank 0 -> rank p */
93                 isend(p, buf, nbytes, 100);
94                 msgwait();
95
96                 /* Pong: rank p -> rank 0 */
97                 irectv(p, buf, nbytes, 101);
98                 msgwait();
99
100             } else if (rank == p) {
101
102                 /* Receive ping from rank 0 */
103                 irectv(0, buf, nbytes, 100);
104                 msgwait();
105
106                 /* Send pong back to rank 0 */
107                 isend(0, buf, nbytes, 101);
108                 msgwait();
109             }
110         }
111     }
112
113     if (rank == 0) {
114         t1 = msg_wtime();
115         return (t1 - t0) / (double)nloop;
116     }
117
118     return 0.0;
119 }

```

We pair the zero rank with all other available ranks, and then output the results to the output file. Note that in the post processing, we find the latency, inverse bandwidth, m_2 , and eager limit. These are computed as follows. All analysis is done on a curve taken from averaging the time for each core. The core-averaged time should hopefully remove any issues caused by some small fluctuations or other issues.

- The latency is calculated by taking a range of very small message sizes (the first 5 messages to be exact) and taking the mean message time across these messages.
- The inverse bandwidth is taken by fitting a line to the last 20% of the data and taking the slope.

Name	Value
Latency	$4.18 \cdot 10^{-7}s$
Inverse bandwidth	$1.06 \cdot 10^{-10}s/\text{bytes}$
m_2	847 bytes
Eager Limit	1184 bytes

Table 1: Caption

- The m_2 value is computed by doing linear interpolation between the first candidate above and first candidate below double the one word time.
- We look through the relative jumps between sequential steps. We then take the largest jump and take the eager-limit transition to happen at the m value right after the largest jump.

Note that the averaging is most important for the eager limit, as the jumps can be sensitive to randomized changes in performance.

2a. Small tests

To do a preliminary test, I tested the code on my Macbook. This was run using 8 cores and with the following result:

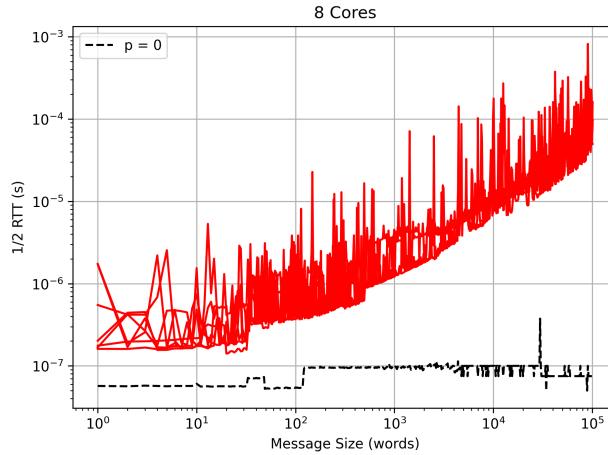


Figure 7: 8 Core Macbook Results

Below are the post processing results:

2b. Larger Tests

I repeated the above analysis for the recommended amounts of MPI ranks and nodes. The results are shown below.

64 Cores, 1 Node

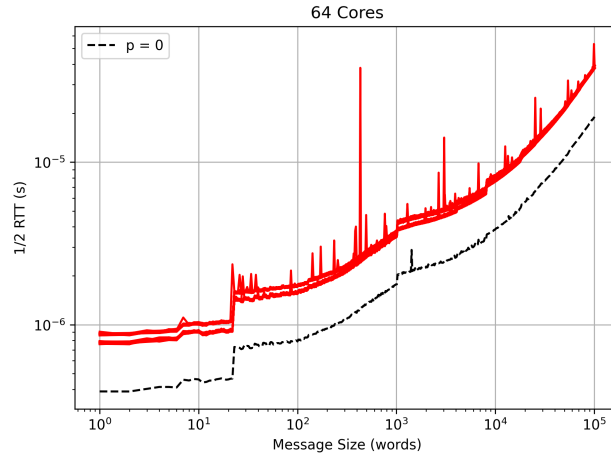


Figure 8: 64 Core Results

Name	Value
Latency	$8.39 \cdot 10^{-7} s$
Inverse bandwidth	$4.13 \cdot 10^{-11} s/\text{bytes}$
m_2	688 bytes
Eager Limit	184 bytes

Table 2: 64 Core Results

128 Cores, 1 Node

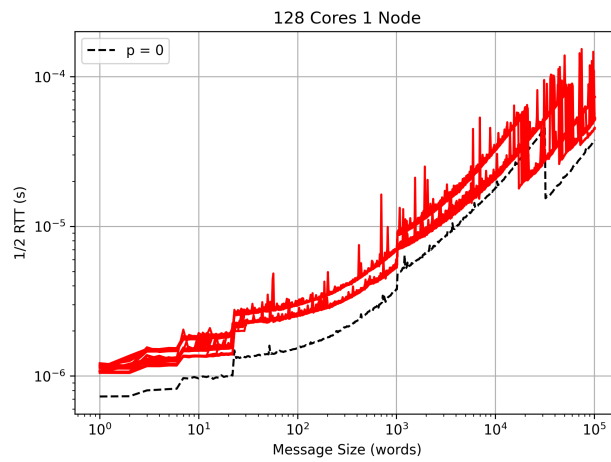


Figure 9: 128 Core, 1 Node Results

Name	Value
Latency	$1.23 \cdot 10^{-6}s$
Inverse bandwidth	$2.75 \cdot 10^{-11}s/\text{bytes}$
m_2	182 bytes
Eager Limit	184 bytes

Table 3: 128 Core, 1 Node Results

128 Cores, 2 Node

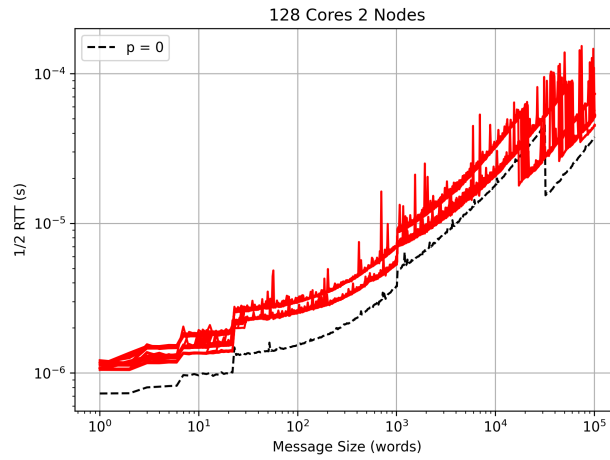


Figure 10: 128 Core, 2 Node Results

Name	Value
Latency	$1.23 \cdot 10^{-6}s$
Inverse bandwidth	$2.75 \cdot 10^{-11}s/\text{bytes}$
m_2	182 bytes
Eager Limit	184 bytes

Table 4: 128 Core, 2 Node Results

192 Cores, 2 Node

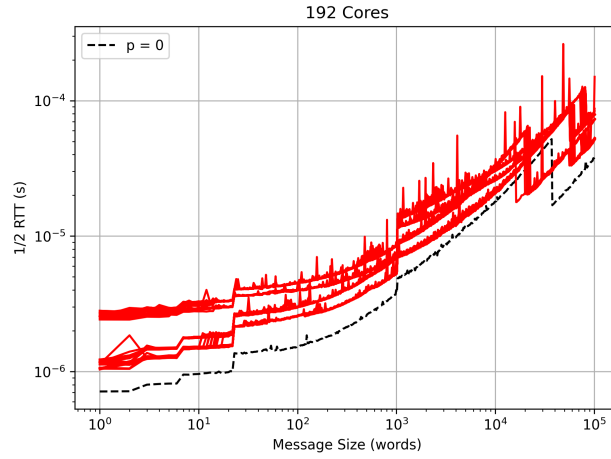


Figure 11: 192 Core Results

Name	Value
Latency	$2.00 \cdot 10^{-6}s$
Inverse bandwidth	$5.03 \cdot 10^{-11}s/\text{bytes}$
m_2	1040 bytes
Eager Limit	8224 bytes

Table 5: 192 Core Results

256 Cores, 2 Node

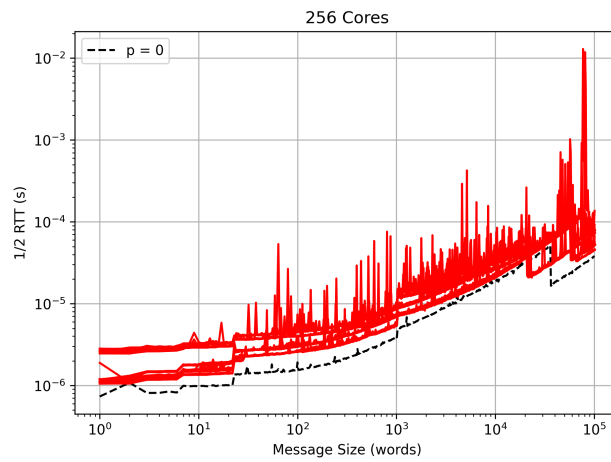


Figure 12: 256 Core Results

Name	Value
Latency	$1.99 \cdot 10^{-6}s$
Inverse bandwidth	$5.24 \cdot 10^{-11}s/\text{bytes}$
m_2	1082 bytes
Eager Limit	617376 bytes

Table 6: 256 Core Results

Discussion

As can be seen, the general shape of the time curves look exactly as we expect. The Latency and inverse bandwidth values are all within the scales that we expect, and the m_2 values seem reasonable. Although the Eager limit value for the lower number of cores seems reasonable, and for the cluster they are consistent, at larger values they break down. I believe this is a post processing issue and not a systems issue. The method of finding the eager limit from the largest jump becomes problematic when certain cores had very large jumps in their computation time, such as in the 256 and 192 core cases. I believe that those large spikes are being detected instead of earlier spikes. In these cases, there is a smaller jump that is more in line with the values of 64, and 128 cores.

In addition, the cluster seems to have optimized away my usage of two cores when taking 128 cores. This is because the results are identical. However, due to time constraints I could not redo the runs.

Appendix

Here, we include the post processing codes for each of the problems

Problem 1

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # =====
5  # Select which attempts to plot
6  # =====
7  attempts = [5]
8
9  # Available markers
10 markers = ['o', 's', '^', 'v', 'D', 'p']
11
12 plt.figure()
13
14 # =====
15 # C implementation attempts
16 # =====
17 for attempt in attempts:
18
19     # Build filename
20     filename = (
21         f"Homework/HW1/Problem1/c_implementation/"
22         f"Attempt{attempt}/benchmark.log"
23     )
24

```

```

25     # Load data
26     data = np.loadtxt(filename)
27
28     N = data[:, 0]
29     gflops = data[:, 2]
30
31     # Find maximum performance
32     imax = np.argmax(gflops)
33
34     print(
35         f"Maximum GFLOPS for Attempt {attempt}: "
36         f"{gflops[imax]:.6f} at N = {N[imax]:.0f}"
37     )
38
39     # Plot
40     plt.plot(
41         N,
42         gflops,
43         marker=markers[(attempt - 1) % len(markers)],
44         label=f"Attempt {attempt}"
45     )
46
47     # =====
48     # MATLAB benchmark
49     # =====
50     matlab_filename = "Homework/HW1/Problem1/matlab_benchmark.log"
51
52     matlab_data = np.loadtxt(matlab_filename)
53
54     N_matlab = matlab_data[:, 0]
55     gflops_matlab = matlab_data[:, 2]
56
57     # Find maximum MATLAB performance
58     imax = np.argmax(gflops_matlab)
59
60     print(
61         f"Maximum GFLOPS for MATLAB: "
62         f"{gflops_matlab[imax]:.6f} at N = {N_matlab[imax]:.0f}"
63     )
64
65     # Always plot MATLAB benchmark
66     plt.plot(
67         N_matlab,
68         gflops_matlab,
69         marker='x',
70         linestyle='--',
71         label="MATLAB"
72     )
73
74     # =====
75     # Plot formatting
76     # =====
77     plt.xlabel("Matrix Size, N")
78     plt.ylabel("Performance (GFLOPS)")
79
80     plt.legend()
81     plt.grid(True)
82     plt.tight_layout()
83
84     # Save figure
85     attempt_string = ''.join(str(attempt) for attempt in attempts)
86     output_filename = f"./Homework/HW1/Problem1/speedPlot_{attempt_string}.png"
87
88     plt.savefig(output_filename, dpi=300, bbox_inches="tight")
89
90     print(f"Figure saved as: {output_filename}")
91
92     plt.show()
93
94     plt.show()

```

Problem 2

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 cases = [
4     ("P256", "256 Cores"),
5     ("P192", "192 Cores"),
6     ("P128_N2", "128 Cores 2 Nodes"),
7     ("P128", "128 Cores 1 Node"),
8     ("P64", "64 Cores"),
9     ("P8", "8 Cores"),
10 ]
11
12 for folder, plotTitle in cases:
13     data = np.loadtxt(f"Homework/HW1/Problem2/cluster_download/HW1P2_RESULTS/{folder}/timing.log", skiprows=1)
14     tit = plotTitle
15
16     p = data[:, 0].astype(int)
17     nwds = data[:, 1]
18     rtt = 0.5*data[:, 4]
19
20     plt.figure()
21
22     for rank in np.unique(p):
23         mask = (p == rank)
24
25         if rank == 0:
26             plt.loglog(
27                 nwds[mask],
28                 rtt[mask],
29                 'k--',
30                 label="p = 0"
31             )
32         else:
33             plt.loglog(
34                 nwds[mask],
35                 rtt[mask],
36                 color = 'red'
37             )
38
39     # =====
40     # Average RTT across all p
41     # =====
42     unique_nwds = np.unique(nwds)
43
44     avg_rtt = np.array([
45         np.mean(rtt[nwds == n])
46         for n in unique_nwds
47     ])
48
49     # plt.loglog(
50     #     unique_nwds,
51     #     avg_rtt,
52     #     linewidth=1.5,
53     #     label="Average",
54     #     color = 'blue'
55     # )
56
57     plt.title(tit)
58     plt.xlabel("Message Size (words)")
59     plt.ylabel("1/2 RTT (s)")
60     plt.legend()
61     plt.grid(True)
62     plt.tight_layout()
63
64     # =====
65     # Latency
66     # =====
67
68     n_small = 5
```

```

69 latency_rtt = np.mean(avg_rtt[:n_small])
70 latency_oneway = latency_rtt / 1.0
71
72
73
74
75 # =====
76 # Inverse bandwidth
77 # =====
78
79 bytes_ = unique_nwds * 8
80
81 # Fit the final 20% of the data
82 n_large = max(2, int(0.2 * len(bytes_)))
83
84 m_large = bytes_[-n_large:]
85 t_large = avg_rtt[-n_large:]
86
87 beta_rtt, alpha_fit = np.polyfit(m_large, t_large, 1)
88
89 beta_oneway = beta_rtt / 1.0
90 bandwidth = 1.0 / beta_oneway
91
92 # =====
93 # Find m2 such that  $t(m2) = 2 * t(1)$ 
94 # =====
95
96 t1 = avg_rtt[0]
97 target_t = 2.0 * t1
98
99 # Find first point where average time reaches/exceeds  $2 * t(1)$ 
100 idx_candidates = np.where(avg_rtt >= target_t)[0]
101
102 if len(idx_candidates) == 0:
103     m2 = np.nan
104     print(f"m2 not reached in available data")
105 else:
106     idx = idx_candidates[0]
107
108     if idx == 0:
109         m2 = unique_nwds[0]
110     else:
111         m1_low = unique_nwds[idx - 1]
112         m1_high = unique_nwds[idx]
113
114         t_low = avg_rtt[idx - 1]
115         t_high = avg_rtt[idx]
116
117         # Linear interpolation
118         m2 = m1_low + (
119             (target_t - t_low)
120             / (t_high - t_low)
121         ) * (m1_high - m1_low)
122
123 # =====
124 # Estimate eager limit
125 # =====
126
127 # Relative change between consecutive average timing points
128 relative_jump = np.diff(avg_rtt) / avg_rtt[:-1]
129
130 # Index of largest relative jump
131 idx_eager = np.argmax(relative_jump)
132
133 # Eager-limit transition occurs between these two message sizes
134 m_before = unique_nwds[idx_eager]
135 m_after = unique_nwds[idx_eager + 1]
136
137 t_before = avg_rtt[idx_eager]
138 t_after = avg_rtt[idx_eager + 1]

```

```

139
140 # Use first message size after the jump as estimate
141 m_eager = m_after
142 print("=====",tit, "=====")
143 print(f"RTT latency      = {latency_rtt:.6e} s")
144 # print(f"One-way latency = {latency_oneway:.6e} s")
145 print(f"RTT inverse bandwidth = {beta_rtt:.6e} s/byte")
146 # print(f"One-way inverse bandwidth = {beta_oneway:.6e} s/byte")
147 # print(f"Bandwidth          = {bandwidth:.6e} bytes/s")
148 # print(f"Bandwidth          = {bandwidth / 1e9:.3f} GB/s")
149 # print(f"t(1)              = {t1:.6e} s")
150 # print(f"2*t(1)            = {target_t:.6e} s")
151 # print(f"m2 where t(m2)=2*t(1) = {m2:.3f} words")
152 print(f"m2                  = {8*m2:.3f} bytes")
153
154 # print(f"Eager limit estimate    = {m_eager:.0f} words")
155 print(f"Eager limit estimate    = {8*m_eager:.0f} bytes")
156 # print(f"Timing jump            = {t_before:.6e} -> {t_after:.6e} s")
157 # print(f"Relative timing jump    = {relative_jump[idx_eager]*100:.2f}%")
158
159 # Save figure
160 plt.savefig(
161     f"./Homework/HW1/Problem2/CorePlot_{tit}.png",
162     dpi=300,
163     bbox_inches="tight"
164 )
165
166 plt.show()

```